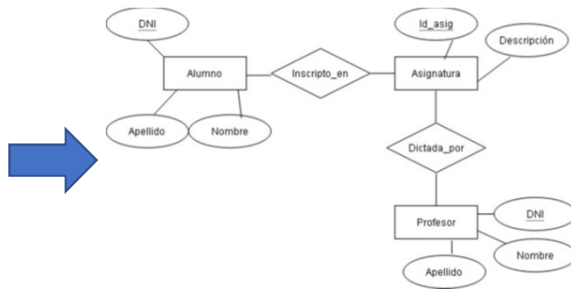


Archivos

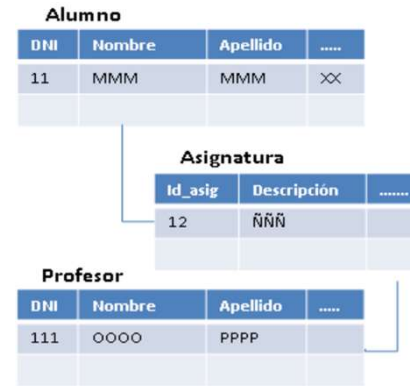
Conceptos generales de organización de Archivos



Mundo Real
-Dominio -



Modelo Conceptual
- DER -



Modelo Lógico
Modelo Relacional
- DT -



Modelo Físico
Los datos de BD están
almacenados
físicamente en un
**medio de
almacenamiento**

ARQUITECTURA DE TRES CAPAS

CAPA APLICACIONES

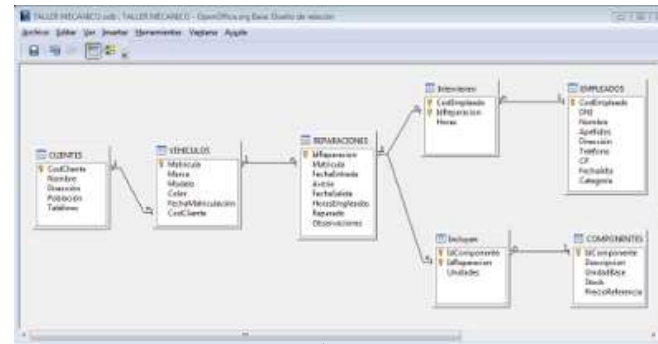
VEHÍCULO		
Matrícula	Modelo	FechaMatricación
K574 PHS	Torres	14/04/07
Marca	Color	CodCliente
Ford	Negro	3

REPARACION	
IdReparacion	FechaReparada
0	25/05/18
Avería	
Cambio amortiguadores	
FechaSolicitud	
12/05/10	
HorasReparadas	Reparado
3,00	☐
Observaciones	

PROPIETARIO	
CodCliente	
2	
Nombre	
Rosario Blanco	
Direccion	
C/ La Alcapala 7	
Poblacion	
Torale	
Telefono	
605999611	

Independencia lógica

CAPA LÓGICA



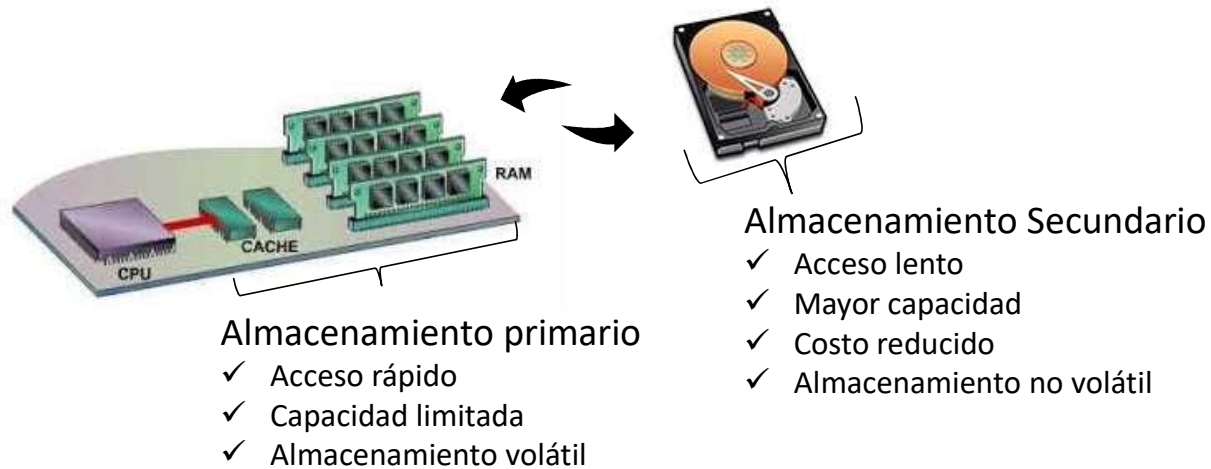
Independencia física

CAPA FÍSICA



Conceptos generales de organización de archivos

Los datos de BD están almacenados físicamente en un **medio de almacenamiento**
El SGDB podrá recuperar, actualizar y procesar estos datos cuando lo requiera



Conceptos generales de organización de archivos

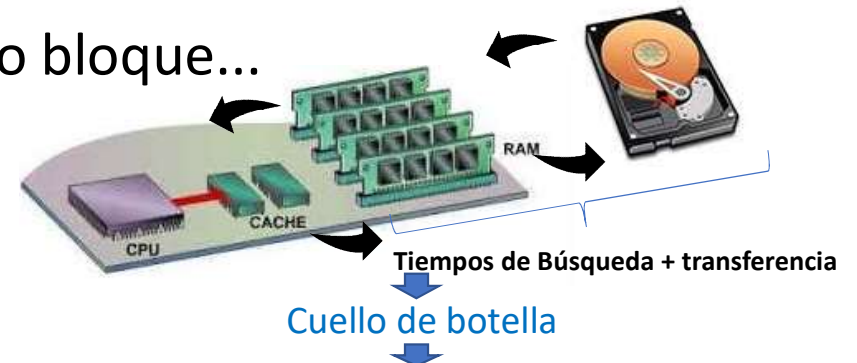
- **Transferencia** de datos entre **memoria principal** y **disco** se realiza en unidades de **bloque**

- Un disco magnético está estructurado en **pistas** y **sectores**
- División de pista en **bloques** (páginas) por el SO al formatear el disco
- El tamaño del bloque se fija durante la inicialización y no puede cambiarse dinámicamente



- Si se necesita información contenida en cierto bloque...

- Se debe conocer:
 - Dirección del bloque
 - Dirección de un **búfer** (memoria intermedia)
- **Lectura**: copia el bloque del disco al búfer
- **Escritura**: copia el bloque del búfer al disco



Estructurar los archivos para **minimizar** el nº de **transferencias de bloques** del disco a memoria principal

Conceptos generales de organización de archivos

- **Datos** en disco están organizados en **registros**
- Un conjunto de registros es un **archivo**
- **Registro**
 - Colección de valores de datos relacionados entre sí, representan a una entidad y sus atributos
 - Cada valor (1 o más bytes) corresponde a un **atributo**
 - Cada campo tiene asociado un **tipo de datos**
 - Definición de **formato o tipo de Registro**: { (nombre-atributo,tipo-datos) }

NOMBRE DEL TIPO DE REGISTRO	NOMBRE DEL CAMPO	TIPO DE DATOS DEL CAMPO (valores que puede aceptar)
type EMPLEADO = record	NOMBRE	: packed array [1..30] of character;
	NSS	: packed array [1..9] of character;
	SALARIO	: integer;
	CÓDIGO_PUESTO	: integer;
	DEPARTAMENTO	: packed array [1..20] of character;
	FECHA_CONTRATO	: packed array [1..4] of character;
end;		

Conceptos generales de organización de archivos

- **Archivo**

- Secuencia de registros, en general todos los registros de un mismo archivo son del mismo tipo.

- Archivo **mixto**

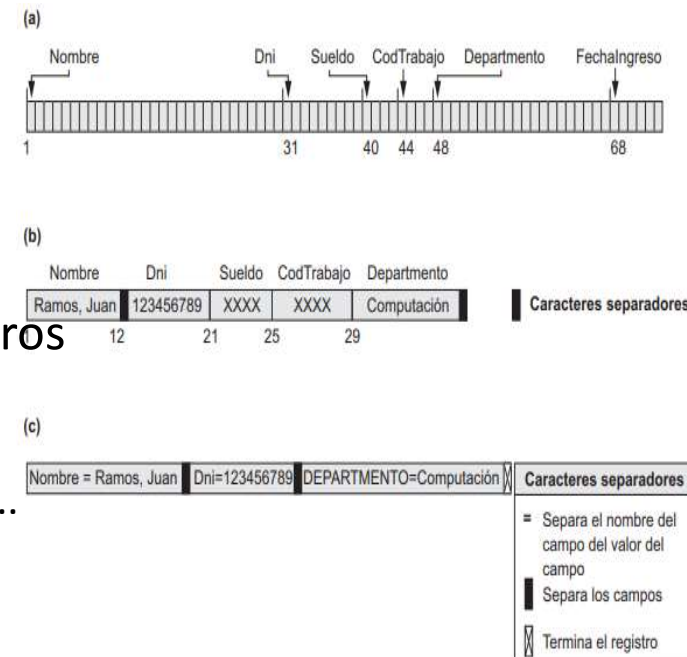
- Contiene registros de diferente tipo
- Aumenta la eficiencia de operaciones de reunión

- Archivo **de registros de longitud fija (a)**, todos los registros

tienen exactamente el mismo tamaño en bytes

- Archivo **de registros de longitud variable (b,c)**, debido a...

- Registros del mismo tipo pero con algún campo ...
 - de longitud variable,
 - multivalorado,
 - opcional
- Archivo mixto



Conceptos generales de organización de Archivos

- Los registros se asignan a bloques de disco

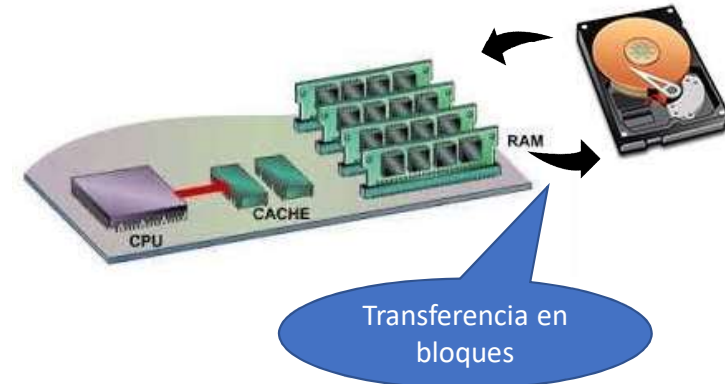
- tamaño(bloque) = B bytes
- b es el número de bloques del archivo

- Archivo con registros de **longitud fija**

- R es la longitud en byte de cada registro
- r es la cantidad de registro del archivo
- Si $B > \text{tamaño(registro)} \Rightarrow$ varios registros por bloque
- Si $B \geq R \Rightarrow$ **factor de bloques fbl** = $\lfloor B / R \rfloor$ registros
- Espacio desocupado = $B - (fbl * R)$ bytes
- **Organización extendida**: aprovechar este espacio almacenando una parte de un registro y el resto en otro bloque



- **Organización no extendida**: no se permite que un registro esté almacenado en varios bloques



Conceptos generales de organización de Archivos

- Los registros se asignan a bloques de disco

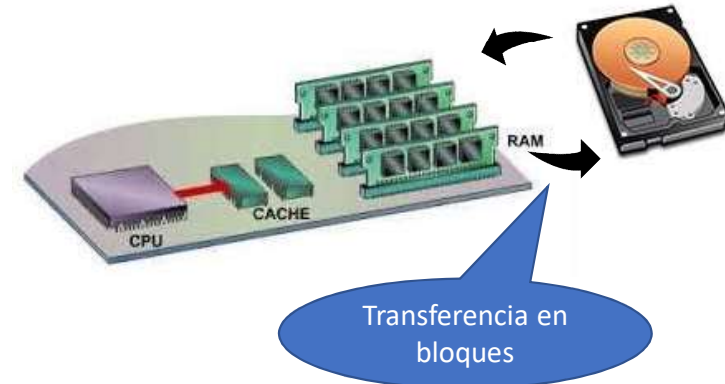
- tamaño(bloque) = B bytes
- b es el número de bloques del archivo

- Archivo con registros de **longitud fija**

- R es la longitud en byte de cada registro
- r es la cantidad de registro del archivo
- Si $B > \text{tamaño(registro)} \Rightarrow$ varios registros por bloque
- Si $B \geq R \Rightarrow$ **factor de bloques fbl** = $\lfloor B / R \rfloor$ registros
- Espacio desocupado = $B - (fbl * R)$ bytes
- **Organización extendida**: aprovechar este espacio almacenando una parte de un registro y el resto en otro bloque



- **Organización no extendida**: no se permite que un registro esté almacenado en varios bloques



Conceptos generales de organización de Archivos

- Los registros se asignan a bloques de disco

- tamaño(bloque) = **B** bytes
- **b** es el número de bloques del archivo

- Archivo con registros de **longitud fija**

- **R** es la longitud en byte de cada registro
- **r** es la cantidad de registro del archivo
- Si **B** > tamaño(registro) $\Rightarrow$ varios registros por bloque
- Si **B** $\geq$ **R** $\Rightarrow$ **factor de bloques fbl** = $\lfloor B / R \rfloor$ registros
- Espacio desocupado = $B - (fbl * R)$ bytes
- **Organización no extendida**: no se permite que un registro esté almacenado en varios bloques



Suponer un disco con **bloques de 512 bytes**, donde se encuentra un archivo que almacena los datos de libros, siendo la longitud total de cada **registro de 102 bytes**

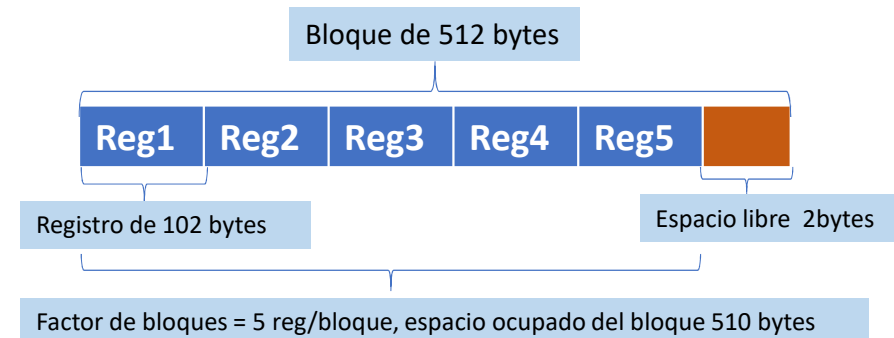
$B = 512$ bytes

$R = 102$ bytes

$$\begin{aligned} fbl &= \lfloor B/R \rfloor = \lfloor 512[\text{bytes/bloque}] / 102 [\text{bytes/reg}] \rfloor \\ &= \lfloor 5.019 [\text{reg/bloque}] \rfloor \\ &= 5 [\text{reg/bloque}] \end{aligned}$$

En un **bloque puedo almacenar 5 registros**, ¿queda espacio libre en cada bloque?

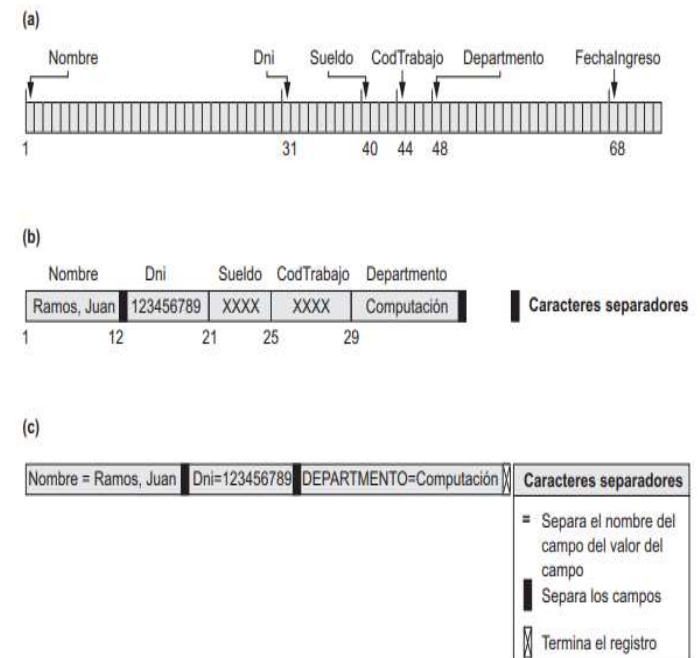
$$\begin{aligned} B - (fbl * R) &= 512 [\text{bytes/bloque}] - (5 [\text{reg/bloque}] * 102 [\text{bytes/reg}]) \\ &= 512 [\text{bytes}] - 510 [\text{bytes}] \\ &= 2 [\text{bytes/bloque}] \end{aligned}$$



Conceptos generales de organización de Archivos

- **Cabecera** de un archivo:

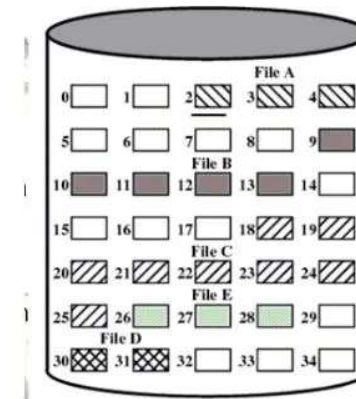
- Contiene datos necesarios para que los programas puedan acceder a los registros del archivo
- Incluye información para **determinar la dirección** en disco de los **bloques** y **descripción de formato** de los registros:
 - Archivo con registros no extendidos y de longitud fija:
 - longitud y orden de campos
 - Archivo con registros de longitud variable:
 - caracteres separadores entre campos
 - marcas de tipo de campo (si campos opcionales)
 - marcas de tipo de registro (si archivo mixto)



Conceptos generales de organización de Archivos

■ Asignación de bloques en disco

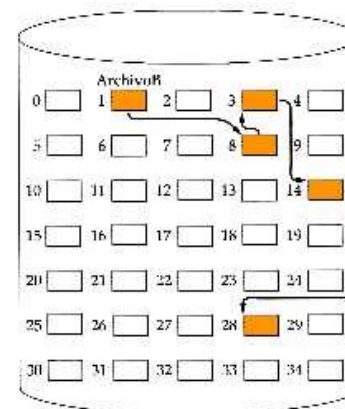
- **Contigua**, los bloques del archivo se asignan a bloques contiguos en el disco



Nombre de Archivo	Bloque inicial	Longitud
Archivo A	2	3
Archivo B	9	5
Archivo C	18	8
Archivo D	30	2
Archivo E	26	3

Cabecera del disco indica en que bloque inicia y termina cada archivo

- **Enlazada**, cada bloque del archivo contiene un puntero al siguiente bloque en ese archivo

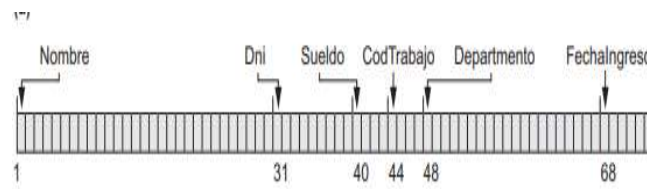


Archivo	Bloque	Longitud
B	1	5

Cabecera del disco indica el puntero al primer bloque de cada archivo

Conceptos generales de organización de Archivos

- Operaciones sobre los Archivos
 - Operaciones de **recuperación** de datos
 - Localizar ciertos registros para examinar y procesar su contenido pero no los cambia
 - Operaciones de **actualización**
 - Insertar, eliminar o modificar registros
- Suele ser necesaria una selección previa con base en una **condición de búsqueda**
 - Si varios registros la satisfacen, se localiza el primero de ellos



Nombre	DNI	Sueldo	CodTrabajo	Departamento	FechaIngreso
Tuli	22222	400	A5	Recepción	01/02/21
Mazzi	33333	500	C7	Compras	12/08/19
Perez	44444	600	C5	Compras	12/05/18
Nazario	66666	400	A5	Recepción	01/02/20

```
SELECT * FROM empleado e WHERE e.dni = 33333;
```

```
SELECT * FROM empleado e WHERE e.departamento = 'Recepción' and e.sueldo >= 300
```

Conceptos generales de organización de Archivos

- **Búsqueda lineal de un registro**, se copia cada registro en el buffer y se examina hasta encontrar el registro, o haber pasado por todos los bloques del archivo sin encontrarlo

- Peor caso leer los b bloques del archivo
- Mejor de los casos encontrar el registro en el primer bloque
- Caso promedio leer $b/2$ bloques



- **Búsqueda binaria de un registro**, los valores de búsqueda están ordenados, se divide al medio el conjunto de valores, y se lo compara con la búsqueda, sino coincide, la mitad en la cual el valor no puede estar se elimina, y se procede de igual modo con la otra mitad, hasta hallar el valor

- Se tienen $\log_2(b)$ accesos a bloques, se encuentre o no el registro
- Mejora a una búsqueda lineal

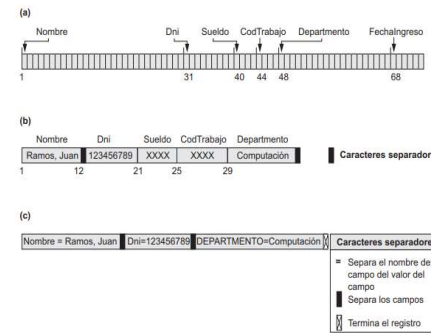


Conceptos generales de organización de Archivos

- **Organización de archivo:**

- Organización de los datos de un archivo

registros



bloques



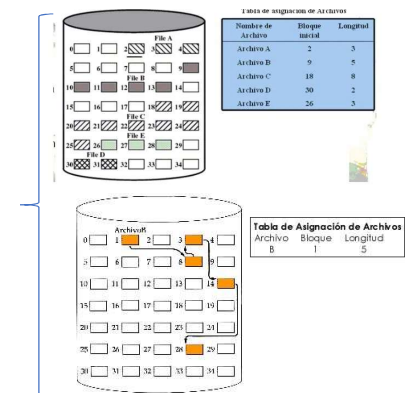
- **Estructuras de acceso**, la forma en que los registros y los bloques se colocan e interconectan en el medio de almacenamiento

- Objetivo: localización eficiente de los datos

- **Método de acceso:**

- Conjunto de operaciones, para recuperación, actualización y búsqueda, que se pueden aplicar a un archivo

Combinados, deben maximizar la **eficiencia** de las **operaciones más frecuentes** sobre el archivo



Conceptos generales de organización de Archivos

Tipos de organización de Archivos

- Según la forma en que los registros del archivo se colocan físicamente en el disco y cómo se puede acceder a ellos
 - **Archivo No Ordenado (montón)**
 - **Archivo Ordenado (secuencial)**
 - **Archivo direccionamiento directo (o hashing)**
- Cuando los registros son accedidos desde una estructura externa o índice Organización indexada
 - **Índice de un nivel**
 - **Índice primario**
 - **Índice secundario**
 - **Índice de agrupamiento**
 - **Índice con indirección**
 - **Índice Multinivel**
 - **Árboles B y B⁺**

Archivos desordenados o de montón

- Los registros se colocan en el orden en el que se insertan.
- Los nuevos registros se insertan al final del archivo.
- Ingresar un nuevo registro** es **eficiente**: se lee el último bloque del archivo en la memoria buffer se agrega el registro y se vuelve a grabar el bloque en el dispositivo y la dirección de este último bloque se registra en la cabecera del archivo.
- Eliminar un registro** implica leer el bloque que tiene el registro en memoria, eliminar (o marcar) el registro y regrabar el bloque en disco.
- El archivo puede requerir una **reorganización periódica** para recuperar espacio y que los bloques ocupen su capacidad.
- Los registros que se empleen en estos archivos pueden ser de **longitud fija o variable**, con **división** de un registro en diferentes bloques o no.
- ¿Qué tipo de búsqueda se puede realizar?*

Ingresar 14

	9				b0
	5				r0
	13				r1
	8				r2
	6				r3
	15				b1
	3				r0
	17				r1
					r2
	21				r3
	11				b2
	16				r0
	2			%	r1
					r2
	24				r3
	10				b3
	20				r0
	1				r1
					r2
	4				r3
	23				b4
	14				r0
					r1
					r2
					r3

Eliminado

Archivos desordenados o de montón

- La búsqueda de un registro dado implica una **búsqueda lineal**. En promedio se leen $b/2$ bloques.
- Para **leer los registros en orden** según un campo se debe crear una copia ordenada de estos registros. La clasificación de un archivo voluminoso puede ser costosa. Se emplea **ordenación externa**.
- Para Archivos de *registros de longitud fija, bloques no extendidos y asignación contigua de bloques* se puede acceder a un registro por su posición relativa, pero no ayuda cuando se busca un registro según una condición.

i -ésimo registro estará en el bloque $\lfloor i/fbl \rfloor$ y será el $(i \bmod fbl)$ -ésimo registro en el bloque

Ejemplo: encontrar el registro de que se encuentra en la 10ma posición

Se ubica en el bloque $\lfloor 10/4 \rfloor = 2$ y será el $(10 \bmod 4) = 2$ registro del bloque

	9				b0
	5				r0
	13				r1
	8				r2
					r3
	6				b1
	15				r0
	3				r1
	17				r2
					r3
	21				b2
	11				r0
	16				r1
	2				r2
					r3
	24				b3
	10				r0
	20				r1
	1				r2
					r3
	4				b4
	23				r0
					r1
					r2
					r3

Archivos secuencial ordenado

- Los registros se **almacenan físicamente ordenados** en el dispositivo de acuerdo con un “campo” de ordenamiento. Si el **campo de ordenamiento** es un campo clave → clave de ordenamiento.

Ventajas respecto a los Archivos no ordenados

- ☐ **Lectura ordenada** muy **eficiente** si el orden es el de los valores del campo de ordenación: **lectura secuencial**
- ☐ **Buscar siguiente** (en el orden del campo de ordenación) no suele necesitar otro acceso a bloque
- ☐ **Buscar** un registro dado su valor del campo de ordenación es rápido: **búsqueda binaria**
 - Las búsquedas que involucran **criterios tales como $<$, $\leq$, $>$, $\geq$** son eficientes porque se conoce el orden de los registros

Pero si la **Lectura ordenada** o **Buscar** no están basados en el campo de ordenación, estas ventajas «desaparecen»

NAME	SSN	BIRTHDATE	JOB	SALARY	SEX
Aaron, Ed					
Abbott, Diane					
⋮					
Accetta, Marc					
⋮					
Adams, John					
Adams, Robin					
⋮					
Akers, Jan					
⋮					
Alexander, Ed					
Alfred, Bob					
⋮					
Allen, Sam					
⋮					
Allen, Troy					
Anders, Keith					
⋮					
Anderson, Rob					
⋮					
Anderson, Zach					
Angell, Joe					
⋮					
Anchor, Sue					

Archivos secuencial ordenado

□ Inserción

- supone **encontrar posición correcta** para el registro y
- abrirle espacio mediante **desplazamiento** de registro, re-escritura de registros (costoso!)

□ Eliminación física o por marca

- con problemas de eficiencia similares a la inserción

□ Modificación

- Si la **condición de búsqueda** se basa en el **campo de ordenación**, usar **búsqueda binaria**. Si no, búsqueda **lineal**
- Si el **campo modificado es el de ordenación**, el nuevo valor puede provocar su **cambio de ubicación**: eliminación + inserción

➤ Para **augmentar la eficiencia de la inserción**

- Dejar espacio libre en cada bloque para nuevos registros
- **Archivo de desbordamiento** auxiliar, no ordenado, que periódicamente se ordena y fusiona con el archivo principal
⇒ **Buscar** se complica: **binaria** en principal + **lineal** en auxiliar

NAME	SSN	BIRTHDATE	JOB	SALARY	SEX
Aaron, Ed					
Abbott, Diane					
⋮					
Acosta, Marc					
⋮					
Adams, John					
Adams, Robin					
⋮					
Akers, Jan					
⋮					
Alexander, Ed					
Alfred, Bob					
⋮					
Allen, Sam					
⋮					
Allen, Troy					
Anders, Keith					
⋮					
Anderson, Rob					
⋮					
Anderson, Zach					
Angell, Joe					
⋮					
Ancher, Sue					

Archivo de datos: Fórmulas de cálculo

Tamaño registro $\rightarrow R$

Tamaño bloque $\rightarrow B$

Cantidad registros del archivo $\rightarrow r$

Factor de bloqueo del archivo $\rightarrow bfl = \lfloor B/R \rfloor$

Cantidad bloques del archivo $\rightarrow b = \lceil r/bfl \rceil$

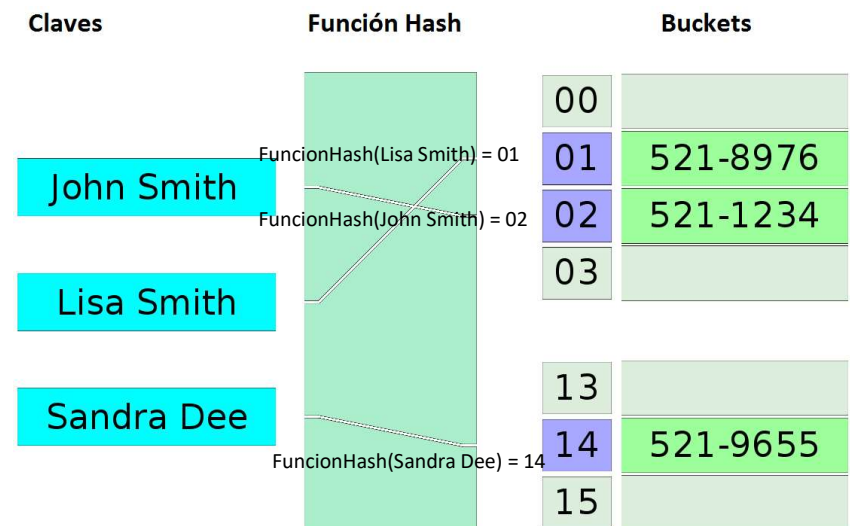
Espacio libre en bloque = $B - (bfl * r)$

Acceso con búsqueda binaria

- Archivo ordenado $\rightarrow \lceil \log_2 b \rceil$
- Archivo no ordenado
 - promedio $\rightarrow b/2$
 - peor caso $\rightarrow b$
 - mejor caso $\rightarrow 1$

Archivos acceso directo (hash)

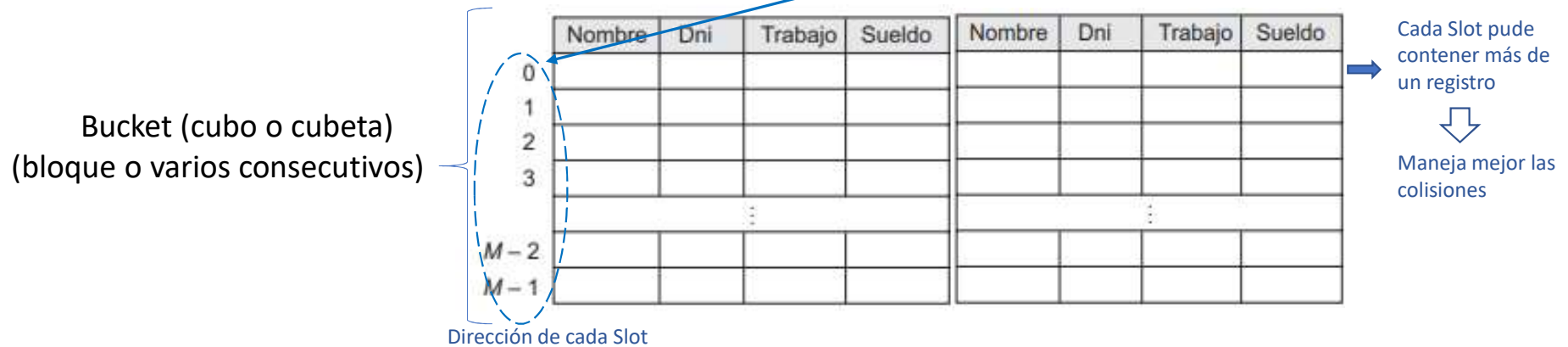
- Se emplea un campo simple (K) del registro denominado **campo hash**, si el campo simple es la clave se denomina **clave hash**.
- Se aplica una **función h** sobre el campo hash que da como resultado la dirección en disco del bloque donde se va a almacenar/recuperar el registro.
 - h** debe ser **sencilla y rápida de calcular**
 - Su **objetivo** es conseguir una **distribución uniforme** de los registros
 - Mínimo número de colisiones
 - Máximo número de posiciones ocupadas



- Este tipo de archivos permite acceso rápido a registros en una búsqueda de igualdad sólo sobre el campo o clave hash.
- En la mayoría de los registros, sólo necesitamos acceder a un bloque para recuperar el registro.

Archivos acceso directo (hash)

La función que se emplea con mayor frecuencia es la del resto de la división: $h(K)=k \bmod M$ (arreglo de 0 a M-1)



- Si el campo hash no es numérico se lo debe transformar para aplicar la función, aunque no es excluyente.
- Este archivo es apto para el acceso de registros al azar, no así para el acceso de registros en orden de acuerdo con la clave.
- Existen funciones hash que “preservan el orden” de las claves.
- Desventaja del direccionamiento calculado o hash
 - **No se garantiza que valores distintos de k produzcan direcciones diferentes en el archivo**

Archivos acceso directo (hash)

- **Colisión:** ocurre cuando el valor de un registro con clave hash diferente da la misma dirección que otro/s registros y debe ser ubicado en un lugar diferente.
- Hallar otro lugar para el nuevo registro cuando se produce una colisión se llama “**resolución de colisiones**”:
 - **Direccionamiento abierto:** A partir de la dirección que da la transformación de la clave hash, el programa busca en las direcciones subsecuentes hasta que encuentra la primera posición vacía.
 - **Encadenamiento:** En este caso se tienen varias direcciones (bloques) aparte donde se guardarán las colisiones. En la primera posición vacía de los bloques se guarda la colisión, y se asigna un puntero hacia esta posición en el bloque que debería haber ocupado. Se mantiene una lista vinculada de colisiones.
 - **Hash múltiple:** se aplica una segunda conversión clave-dirección, si da una nueva colisión se aplica una tercera ó alguno de los métodos para colisiones descritos anteriormente.

Archivos acceso directo (hash)

Colisión: ocurre cuando el valor de un registro con clave hash diferente da la misma dirección que otro/s registros y debe ser ubicado en un lugar diferente. Hallar otro lugar para el nuevo registro cuando se produce una colisión se llama “**resolución de colisiones**”:

Direccionamiento abierto: A partir de la dirección que da la transformación de la clave hash, el programa busca en las direcciones subsecuentes hasta que encuentra la primera posición vacía.

Se la tabla hash con 7 cubetas de 1 registro cada una. Cada registro tiene los siguientes campos: Código, Número, Deporte. La función hash se llama “h”. Se dese almacenar los siguientes registros, en caso de colisión, utilizar direccionamiento abierto o derrame consecutivo, muestre cómo quedaría la tabla resultante

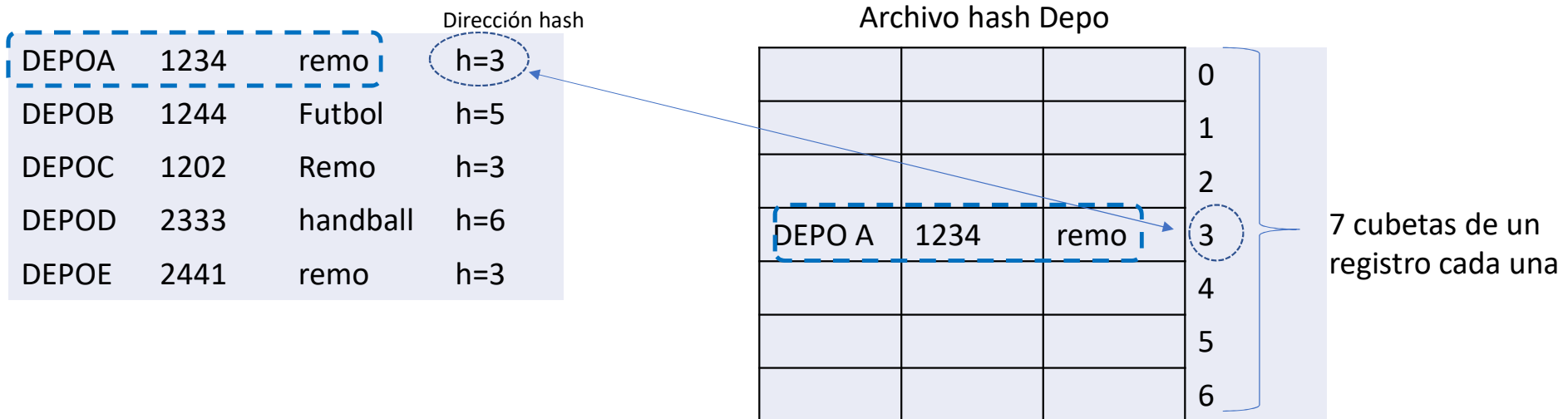
			Dirección hash
DEPOA	1234	remo	h=3
DEPOB	1244	Futbol	h=5
DEPOC	1202	Remo	h=3
DEPOD	2333	handball	h=6
DEPOE	2441	remo	h=3

Archivos acceso directo (hash)

Colisión: ocurre cuando el valor de un registro con clave hash diferente da la misma dirección que otro/s registros y debe ser ubicado en un lugar diferente. Hallar otro lugar para el nuevo registro cuando se produce una colisión se llama “**resolución de colisiones**”:

Direccionamiento abierto: A partir de la dirección que da la transformación de la clave hash, el programa busca en las direcciones subsecuentes hasta que encuentra la primera posición vacía.

Se la tabla hash con 7 cubetas de 1 registro cada una. Cada registro tiene los siguientes campos: Código, Número, Deporte. La función hash se llama “h”. Se dese almacenar los siguientes registros, en caso de colisión, utilizar direccionamiento abierto o derrame consecutivo, muestre cómo quedaría la tabla resultante



Archivos acceso directo (hash)

Colisión: ocurre cuando el valor de un registro con clave hash diferente da la misma dirección que otro/s registros y debe ser ubicado en un lugar diferente. Hallar otro lugar para el nuevo registro cuando se produce una colisión se llama “**resolución de colisiones**”:

Direccionamiento abierto: A partir de la dirección que da la transformación de la clave hash, el programa busca en las direcciones subsecuentes hasta que encuentra la primera posición vacía.

Se la tabla hash con 7 cubetas de 1 registro cada una. Cada registro tiene los siguientes campos: Código, Número, Deporte. La función hash se llama “h”. Se dese almacenar los siguientes registros, en caso de colisión, utilizar direccionamiento abierto o derrame consecutivo, muestre cómo quedaría la tabla resultante

Dirección hash			
DEPOA	1234	remo	h=3
DEPOB	1244	Futbol	h=5
DEPOC	1202	Remo	h=3
DEPOD	2333	handball	h=6
DEPOE	2441	remo	h=3

Archivo hash Depo			
			0
			1
			2
DEPO A	1234	remo	3
			4
DEPOB	1244	Futbol	5
			6

7 cubetas de un registro cada una

Archivos acceso directo (hash)

Colisión: ocurre cuando el valor de un registro con clave hash diferente da la misma dirección que otro/s registros y debe ser ubicado en un lugar diferente. Hallar otro lugar para el nuevo registro cuando se produce una colisión se llama “**resolución de colisiones**”:

Direccionamiento abierto: A partir de la dirección que da la transformación de la clave hash, el programa busca en las direcciones subsecuentes hasta que encuentra la primera posición vacía.

Se la tabla hash con 7 cubetas de 1 registro cada una. Cada registro tiene los siguientes campos: Código, Número, Deporte. La función hash se llama “h”. Se dese almacenar los siguientes registros, en caso de colisión, utilizar direccionamiento abierto o derrame consecutivo, muestre cómo quedaría la tabla resultante

			Dirección hash
DEPOA	1234	remo	h=3
DEPOB	1244	Futbol	h=5
DEPOC	1202	Remo	h=3
DEPOD	2333	handball	h=6
DEPOE	2441	remo	h=3

			Archivo hash Depo	
1er slot libre				0
				1
				2
	DEPO A	1234	remo	3
	DEPO A	1234	remo	4
	DEPOB	1244	Futbol	5
				6
				7 cubetas de un registro cada una

Archivos acceso directo (hash)

Colisión: ocurre cuando el valor de un registro con clave hash diferente da la misma dirección que otro/s registros y debe ser ubicado en un lugar diferente. Hallar otro lugar para el nuevo registro cuando se produce una colisión se llama “**resolución de colisiones**”:

Direccionamiento abierto: A partir de la dirección que da la transformación de la clave hash, el programa busca en las direcciones subsecuentes hasta que encuentra la primera posición vacía.

Se la tabla hash con 7 cubetas de 1 registro cada una. Cada registro tiene los siguientes campos: Código, Número, Deporte. La función hash se llama “h”. Se dese almacenar los siguientes registros, en caso de colisión, utilizar direccionamiento abierto o derrame consecutivo, muestre cómo quedaría la tabla resultante

Dirección hash			
DEPOA	1234	remo	h=3
DEPOB	1244	Futbol	h=5
DEPOC	1202	Remo	h=3
DEPOD	2333	handball	h=6
DEPOE	2441	remo	h=3

Archivo hash Depo			
DEPOE	2441	remo	0
			1
			2
DEPO A	1234	remo	3
DEPOC	1202	remo	4
DEPB	1244	futbol	5
DEPOD	2333	handball	6

7 cubetas de un registro cada una

Archivos acceso directo (hash)

Colisión: ocurre cuando el valor de un registro con clave hash diferente da la misma dirección que otro/s registros y debe ser ubicado en un lugar diferente. Hallar otro lugar para el nuevo registro cuando se produce una colisión se llama “**resolución de colisiones**”

Encadenamiento: En este caso se tienen varias direcciones (bloques) aparte donde se guardarán las colisiones. En la primera posición vacía de los bloques se guarda la colisión, y se asigna un puntero hacia esta posición en el bloque que debería haber ocupado. Se mantiene una lista vinculada de colisiones.

Se la tabla hash con 7 cubetas de 1 registro cada una. Cada registro tiene los siguientes campos: Código, Número, Deporte. La función hash se llama “h”. Se dese almacenar los siguientes registros, en caso de colisión, utilizar direccionamiento abierto o derrame consecutivo, muestre cómo quedaría la tabla resultante

Dirección hash			
DEPOA	1234	remo	h=3
DEPOB	1244	Futbol	h=5
DEPOC	1202	Remo	h=3
DEPOD	2333	handball	h=6
DEPOE	2441	remo	h=3

Archivos acceso directo (hash)

Colisión: ocurre cuando el valor de un registro con clave hash diferente da la misma dirección que otro/s registros y debe ser ubicado en un lugar diferente. Hallar otro lugar para el nuevo registro cuando se produce una colisión se llama “**resolución de colisiones**”

Encadenamiento: En este caso se tienen varias direcciones (bloques) aparte donde se guardarán las colisiones. En la primera posición vacía de los bloques se guarda la colisión, y se asigna un puntero hacia esta posición en el bloque que debería haber ocupado. Se mantiene una lista vinculada de colisiones.

Se la tabla hash con 7 cubetas de 1 registro cada una. Cada registro tiene los siguientes campos: Código, Número, Deporte. La función hash se llama “h”. Se dese almacenar los siguientes registros, en caso de colisión, utilizar direccionamiento abierto o derrame consecutivo, muestre cómo quedaría la tabla resultante

resultante

			Dirección hash
DEPOA	1234	remo	h=3
DEPOB	1244	Futbol	h=5
DEPOC	1202	Remo	h=3
DEPOD	2333	handball	h=6
DEPOE	2441	remo	h=3

Archivo hash Depo

			0
			1
			2
DEPOA	1234	remo	3
			4
DEPOB	1244	Futbol	5
			6

[illegible]

Archivos acceso directo (hash)

Colisión: ocurre cuando el valor de un registro con clave hash diferente da la misma dirección que otro/s registros y debe ser ubicado en un lugar diferente. Hallar otro lugar para el nuevo registro cuando se produce una colisión se llama “**resolución de colisiones**”

Encadenamiento: En este caso se tienen varias direcciones (bloques) aparte donde se guardarán las colisiones. En la primera posición vacía de los bloques se guarda la colisión, y se asigna un puntero hacia esta posición en el bloque que debería haber ocupado. Se mantiene una lista vinculada de colisiones.

Sea la tabla hash con 7 cubetas de 1 registro cada una. Cada registro tiene los siguientes campos: Código, Número, Deporte. La función hash se llama “h”. Se dese almacenar los siguientes registros, en caso de colisión, utilizar direccionamiento abierto o derrame consecutivo, muestre cómo quedaría la tabla resultante

